UNIVERSIDAD NACIONAL DE LUJÁN

Departamento de Ciencias Básicas • Lic. en Sistemas

Programación Orientada a Objetos (11[0|2]76)

Clase 05 • **Ejercicio 03** • Ciclo Lectivo 2026

MATERIAL PARA ESTUDIANTES
ENUNCIADO PRÁCTICO

Ejercicio 3: Mercado de Divisas, Cotizaciones y Valuación con TDD

Contexto

En este ejercicio aplicaremos el ciclo **Test-Driven Development (TDD)** para modelar la **colaboración entre objetos mediante el paso de referencias por constructor**, asegurando que los cálculos se mantengan acoplados al estado dinámico actualizado sin duplicar datos redundantes (*stale data*), e integrando **Java Streams** en la fase de refactorización.

Enunciado

Un **Mercado Financiero** (`Mercado`) provee cotizaciones dinámicas de varias divisas (como `"ARS"` , `"EUR"`) respecto al dólar estadounidense (`USD`).

Por otro lado, los inversores administran un **Portafolio** (`Portafolio`) que acumula tenencias de diversas divisas. Se requiere calcular en cualquier momento el valor total acumulado del portafolio expresado en dólares (`USD`).

Reglas de negocio e invariantes:

1. **Cotizaciones Válidas:** El `Mercado` solo acepta cotizaciones estrictamente positivas (> 0), rechazando valores ≤ 0 con `IllegalArgumentException` .

2. **Referencia Colaboradora por Constructor:** El `Portafolio` debe recibir la referencia a su `Mercado` colaborador obligatoriamente en el constructor. Si se pasa `null`, debe lanzarse `IllegalArgumentException`.
3. **Posiciones Válidas:** Solo se pueden agregar cantidades positivas (> 0) a las posiciones del portafolio.
4. **Reactividad Dinámica (Single Source of Truth):** El `Portafolio` no debe almacenar cotizaciones fijas en sus posiciones. Si el mercado actualiza una cotización, la valuación del portafolio debe reflejar el nuevo valor en tiempo real sin requerir cambios en el portafolio.
5. **Divisas sin Cotización:** Si el portafolio posee una divisa que el mercado no cotiza, el cálculo debe abortar lanzando `CotizacionNoEncontradaException`.

Tareas:

1. Diseñe mediante TDD las clases `Mercado`, `Portafolio` y la excepción `CotizacionNoEncontradaException`.
2. Implemente en `Mercado`:
    - `actualizarCotizacion(String divisa, double valorEnUSD)`: Registra o actualiza la cotización.
    - `getCotizacion(String divisa)`: Retorna la cotización o lanza `CotizacionNoEncontradaException`.
3. Implemente en `Portafolio`:
    - `agregarPosicion(String divisa, double cantidad)`: Acumula tenencia de una divisa.
    - `getPosicion(String divisa)`: Retorna la tenencia actual de la divisa solicitada.
    - `calcularValorTotalUSD()`: Calcula y retorna la sumatoria total en USD multiplicando cada tenencia por la cotización en vivo del mercado.
4. Escriba pruebas unitarias con JUnit 5 avanzando incrementalmente desde las cotizaciones individuales hasta la reactividad ante cambios del mercado y el manejo de divisas desconocidas.

Restricciones y Tips

- **Paso de Referencias:** La referencia a la instancia de `Mercado` debe suministrarse desde el exterior a través del constructor. El `Portafolio` jamás debe instanciar un `new Mercado()` internamente ni almacenar copias fijas de cotizaciones.
- **El Refactor hacia Java Streams:** Al implementar `calcularValorTotalUSD()`, aproveche la fase de Refactor para escribir una consulta declarativa con Streams (`mapToDouble(...).sum()`), eliminando acumuladores mutables procedimentales.
- **El Test de Reactividad:** Diseñe un test donde actualice la cotización en el `Mercado` y aserte inmediatamente que el `Portafolio` recalcula el total correctamente sin haber invocado ningún método mutador sobre él.

Preguntas Conceptuales

Al finalizar el ejercicio, reflexione sobre las siguientes preguntas:

1. ¿Por qué pasar la instancia de `Mercado` por el constructor de `Portafolio` es superior a hacer que `Portafolio` cree su propio mercado interno con `new Mercado()`?
2. ¿Qué ocurre con la consistencia del sistema si duplicáramos el valor de la cotización dentro de la clase `Portafolio` en lugar de delegar la consulta al `Mercado`?
3. ¿Qué ventajas ofrece la API de Java Streams (`mapToDouble().sum()`) frente a bucles tradicionales con acumuladores mutables para el cálculo de valuación?
4. ¿Por qué ante la ausencia de cotización de una divisa es preferible lanzar una excepción de dominio (`CotizacionNoEncontradaException`) en lugar de asumir un valor por defecto (como `0.0`) o ignorar silenciosamente la posición?